

MARAN PMS — Prompt 54: Cajas — Mejoras para Producción

Contexto

El sistema de cajas ya funciona bien. Tiene:

- 4 áreas: recepción, restaurant, spa, eventos
- Turnos con apertura/cierre, arqueo de efectivo, diferencias
- Movimientos manuales e ingresos automáticos desde cada módulo
- Reporte de turno imprimible (`printClosingSummary`)
- Historial de turnos cerrados
- Anulación de movimientos con motivo

Archivos

- `client/src/pages/cash-register.tsx` — todo el frontend de cajas
- `server/routes/cash.ts` (o donde estén las rutas de caja)
- `server/storage.ts` — `closeShift` , `registerCashMovement` , `getCashSummary`

Lo que falta para producción

MEJORA 1 — Reporte de turno: incluir anulaciones y detalle completo

Problema actual: `printClosingSummary()` muestra movimientos y totales por método, pero no muestra anulaciones ni los cambios de reservas del día que sí aparecen en el dialog.

Fix en `printClosingSummary` **en** `cash-register.tsx` :

Agregar sección de anulaciones al HTML del reporte:

```
// Filtrar movimientos anulados para mostrarlos aparte
const anulados = movements.filter(m => m.anulado);
const activos = movements.filter(m => !m.anulado);

// En el HTML, después de la tabla de movimientos activos, agregar:
const anulSection = anulados.length > 0 ? `
  <h2>Movimientos Anulados (${anulados.length})</h2>
  <table>
    <thead>
      <tr>
        <th>Hora</th><th>Descripción</th><th>Método</th><th>Monto</th><th>Motivo
      </tr>
    </thead>
    <tbody>
      ${anulados.map(m => `
        <tr style="color:#999;text-decoration:line-through">
          <td>${formatTime(m.createdAt)}</td>
          <td>${m.description || m.sourceLabel || '-'}</td>
          <td>${PAYMENT_METHOD_MAP[m.paymentMethod] || m.paymentMethod}</td>
          <td>${formatCurrency(Math.abs(parseFloat(String(m.amount))))}</td>
          <td>${m.motivoAnulacion || '-'}</td>
        </tr>
      `).join('')}
    </tbody>
  </table>
  ` : '';

// Agregar también el resumen de diferencia de efectivo:
const diferenciaSection = `
  <div style="margin-top:16px;padding:12px;border:2px solid ${Math.abs(diferencia)
    <p><strong>Efectivo sistema:</strong> ${formatCurrency(efectivoSistema)}</p>
    <p><strong>Efectivo contado:</strong> ${formatCurrency(efectivoContado)}</p>
    <p style="font-size:16px;font-weight:bold;color:${diferencia !== 0 ? '#e53e3e'
      Diferencia: ${diferencia > 0 ? '+' : ''}${formatCurrency(diferencia)}
    </p>
  </div>
  `;
```

Pasar efectivoSistema , efectivoContado **y** diferencia **como parámetros adicionales a** printClosingSummary :

```
// Firma actualizada:
function printClosingSummary(
  shift: CashShift,
```

```

    movements: CashMovement[],
    efectivoSistema: number,
    efectivoContado: number
  ) { ... }

// Llamada actualizada:
onClick={() => printClosingSummary(
  closingSummaryData.shift,
  closingSummaryData.movements,
  efectivoSistema,
  efectivoContado
)}

```

MEJORA 2 — Tab “Resumen del Día” — todas las cajas juntas

Problema: hoy no existe una vista consolidada de todas las cajas para el día. Hay que ir tab por tab para ver cómo va cada área.

Agregar un nuevo tab “Resumen del Día” en `CashRegister`:

```

// En TabsList, agregar después de los tabs de área y antes de Historial:
<TabsTrigger value="resumen-dia" data-testid="tab-resumen-dia">
  <BarChart3 className="h-4 w-4 mr-1" />
  Resumen del Día
</TabsTrigger>

// TabsContent:
<TabsContent value="resumen-dia">
  <ResumenDiaTab />
</TabsContent>

```

Componente `ResumenDiaTab`:

```

function ResumenDiaTab() {
  const today = new Date().toLocaleDateString("en-CA", {
    timeZone: "America/Argentina/Buenos_Aires"
  });
  const [selectedDate, setSelectedDate] = useState(today);

  // Usar el endpoint caja-unificada que ya existe
  const { data: unificada, isLoading } = useQuery<any>({
    queryKey: ["/api/reports/caja-unificada", selectedDate],

```

```

    queryFn: async () => {
      const res = await fetch(
        `/api/reports/caja-unificada?fecha=${selectedDate}`,
        { credentials: "include" }
      );
      if (!res.ok) throw new Error("Error cargando reporte");
      return res.json();
    },
  });

// También traer los turnos del día por área para ver estado de cierre
const { data: turnosDia = [] } = useQuery<any[]>({
  queryKey: ["/api/cash/summary", `?from=${selectedDate}&to=${selectedDate}`],
  queryFn: async () => {
    const res = await fetch(
      `/api/cash/summary?from=${selectedDate}&to=${selectedDate}`,
      { credentials: "include" }
    );
    if (!res.ok) return [];
    return res.json();
  },
});

// Agrupar turnos del día por área
const turnosPorArea = turnosDia.reduce((acc: any, t: any) => {
  if (!acc[t.area]) acc[t.area] = [];
  acc[t.area].push(t);
  return acc;
}, {});

const areas = ["reception", "restaurant", "spa", "events"];
const areaLabels: Record<string, string> = {
  reception: "Recepción",
  restaurant: "Restaurante",
  spa: "SPA",
  events: "Eventos",
};

if (isLoading) return <div className="p-8 text-center"><Skeleton className="h-6

const porModulo = unificada?.porModulo || {};
const porMetodo = unificada?.porMetodo || {};
const totalGeneral = Object.values(porMetodo as Record<string, number>)
  .reduce((s: number, v: any) => s + parseFloat(String(v || 0)), 0);

return (
  <div className="space-y-6 p-4">

```

```

{/* Selector de fecha */}
<div className="flex items-center gap-3">
  <Label>Fecha</Label>
  <Input
    type="date"
    value={selectedDate}
    onChange={(e) => setSelectedDate(e.target.value)}
    className="w-44"
    data-testid="input-resumen-fecha"
  />
  {selectedDate === today && (
    <Badge variant="outline" className="text-green-600 border-green-400">Ho
  )}
</div>

```

```

{/* Tarjetas por área */}
<div className="grid grid-cols-2 md:grid-cols-4 gap-4">
  {areas.map(area => {
    const areaData = porModulo[area] || { total: 0, transacciones: 0 };
    const turnosArea = turnosPorArea[area] || [];
    const hayTurnoAbierto = turnosArea.some((t: any) => t.status === "open")

    return (
      <Card key={area} className={`border-l-4 ${
        area === "reception" ? "border-l-blue-400" :
        area === "restaurant" ? "border-l-orange-400" :
        area === "spa" ? "border-l-purple-400" :
        "border-l-green-400"
      }`} >
        <CardContent className="p-4">
          <div className="flex items-center justify-between mb-2">
            <p className="text-sm font-medium text-muted-foreground">
              {areaLabels[area]}
            </p>
            <Badge
              variant={hayTurnoAbierto ? "default" : "secondary"}
              className="text-xs"
            >
              {hayTurnoAbierto ? "Abierta" : "Cerrada"}
            </Badge>
          </div>
          <p className="text-2xl font-bold">
            {formatCurrency(parseFloat(String(areaData.total || 0)))}
          </p>
          <p className="text-xs text-muted-foreground mt-1">
            {areaData.transacciones || 0} transacciones
          </p>
        </CardContent>
      </Card>
    )
  })}
</div>

```

```

        </CardContent>
    </Card>

    );
    }}}
</div>

{ /* Consolidado por método de pago */
<Card>
    <CardHeader className="pb-3">
        <CardTitle className="text-base flex items-center gap-2">
            <CreditCard className="h-4 w-4" />
            Consolidado por Método de Pago
        </CardTitle>
    </CardHeader>
    <CardContent>
        {Object.keys(porMetodo).length === 0 ? (
            <p className="text-sm text-muted-foreground text-center py-4">
                Sin movimientos para esta fecha
            </p>
        ) : (
            <Table>
                <TableHeader>
                    <TableRow>
                        <TableHead>Método</TableHead>
                        <TableHead className="text-right">Total</TableHead>
                        <TableHead className="text-right">% del día</TableHead>
                    </TableRow>
                </TableHeader>
                <TableBody>
                    {Object.entries(porMetodo as Record<string, number>)
                        .sort(([a], [b]) => b - a)
                        .map([metodo, total]) => (
                            <TableRow key={metodo}>
                                <TableCell>
                                    {PAYMENT_METHOD_MAP[metodo] || metodo}
                                </TableCell>
                                <TableCell className="text-right font-medium">
                                    {formatCurrency(parseFloat(String(total)))}
                                </TableCell>
                                <TableCell className="text-right text-muted-foreground text
                                    {totalGeneral > 0
                                        ? `\${((parseFloat(String(total)) / totalGeneral) * 100)
                                        : '0%'
                                    }
                                </TableCell>
                            </TableRow>
                        ))
                }
            </Table>
        )
    }
}

```

```

    }
    <TableRow className="font-bold border-t-2">
      <TableCell>TOTAL GENERAL</TableCell>
      <TableCell className="text-right text-lg">
        {formatCurrency(totalGeneral)}
      </TableCell>
      <TableCell className="text-right">100%</TableCell>
    </TableRow>
  </TableBody>
</Table>
  )}
</CardContent>
</Card>

{/* Botón imprimir resumen diario */}
{totalGeneral > 0 && (
  <div className="flex justify-end">
    <Button
      variant="outline"
      onClick={() => printResumenDia(selectedDate, porModulo, porMetodo, to
        data-testid="btn-print-resumen-dia"
    >
      <Printer className="h-4 w-4 mr-2" />
      Imprimir Resumen del Día
    </Button>
  </div>
  )}
</div>
);
}

```

Función printResumenDia :

```

function printResumenDia(
  fecha: string,
  porModulo: Record<string, any>,
  porMetodo: Record<string, number>,
  totalGeneral: number,
  areaLabels: Record<string, string>
) {
  const fechaDisplay = new Date(fecha + "T12:00:00")
    .toLocaleDateString("es-AR", { weekday: "long", day: "2-digit", month: "long"

  const html = `<!DOCTYPE html><html><head>
    <title>Resumen Diario - ${fechaDisplay}</title>
    <style>

```

```

body { font-family: Arial, sans-serif; padding: 24px; max-width: 700px; mar
h1 { font-size: 20px; margin-bottom: 4px; }
h2 { font-size: 14px; color: #666; margin-bottom: 20px; }
h3 { font-size: 13px; margin: 20px 0 8px; border-bottom: 1px solid #ccc; pa
table { width: 100%; border-collapse: collapse; margin: 12px 0; font-size:
th, td { border: 1px solid #ddd; padding: 7px 10px; text-align: left; }
th { background: #f5f5f5; font-weight: bold; }
.total { font-weight: bold; background: #eee; font-size: 13px; }
.right { text-align: right; }
.footer { text-align: center; margin-top: 24px; font-size: 10px; color: #99
</style>
</head><body>
<h1>Maran Suites & Towers</h1>
<h2>Resumen Diario de Cajas - ${fechaDisplay}</h2>

<h3>Por Área</h3>
<table>
  <thead><tr><th>Área</th><th class="right">Total</th><th class="right">Trans
  <tbody>
    ${Object.entries(porModulo).map(([area, data]: [string, any]) => `
      <tr>
        <td>${areaLabels[area] || area}</td>
        <td class="right">${formatCurrency(parseFloat(String(data.total || 0)
        <td class="right">${data.transacciones || 0}</td>
      </tr>
    `).join('')}
    <tr class="total">
      <td>TOTAL</td>
      <td class="right">${formatCurrency(totalGeneral)}</td>
      <td class="right">${Object.values(porModulo).reduce((s: number, d: any)
    </tr>
  </tbody>
</table>

<h3>Por Método de Pago</h3>
<table>
  <thead><tr><th>Método</th><th class="right">Total</th><th class="right">%</
  <tbody>
    ${Object.entries(porMetodo)
      .sort(([a], [b]) => b - a)
      .map(([metodo, total]) => `
        <tr>
          <td>${PAYMENT_METHOD_MAP[metodo] || metodo}</td>
          <td class="right">${formatCurrency(parseFloat(String(total)))}</td>
          <td class="right">${totalGeneral > 0 ? ((parseFloat(String(total))
        </tr>
      `).join('')}

```



```

        <tr class="total">
            <td>TOTAL GENERAL</td>
            <td class="right">${formatCurrency(totalGeneral)}</td>
            <td class="right">100%</td>
        </tr>
    </tbody>
</table>

<div class="footer">
    Generado el ${new Date().toLocaleString("es-AR")} | Maran Suites & Towers
</div>
<script>window.onload = function() { window.print(); };</script>
</body></html>`;

const w = window.open("", "_blank");
if (w) { w.document.write(html); w.document.close(); }
}

```

MEJORA 3 — Ajustar endpoint `/api/reports/caja-unificada` para devolver por módulo

El endpoint ya existe pero devuelve `porMetodo` solamente. Agregar `porModulo` y `totalGeneral`:

En `server/routes/reports.ts` (o donde esté el endpoint), ampliar la respuesta:

```

// Al final del endpoint, antes de res.json():
const porModulo: Record<string, { total: number; transacciones: number }> = {};

for (const p of todos) {
    const mod = (p as any).modulo || "otros";
    if (!porModulo[mod]) porModulo[mod] = { total: 0, transacciones: 0 };
    porModulo[mod].total += parseFloat((p as any).monto || "0");
    porModulo[mod].transacciones += 1;
}

res.json({
    fecha,
    movimientos: todos,          // detalle completo (ya existe)
    porMetodo,                   // por método de pago (ya existe)
    porModulo,                   // ← NUEVO: por área/módulo
    totalGeneral: Object.values(porMetodo).reduce((s, v) => s + v, 0),
});

```

MEJORA 4 — Bloquear edición de movimientos en turno cerrado

Problema: hoy se puede anular un movimiento de un turno ya cerrado.

Fix en el backend (en el endpoint `PATCH /api/cash/movements/:id/anular`):

```
// Verificar que el turno esté abierto antes de anular
const movement = await db.select().from(cashMovements)
  .where(eq(cashMovements.id, req.params.id));

if (!movement[0]) return res.status(404).json({ error: "Movimiento no encontrado"

if (movement[0].shiftId) {
  const shift = await db.select().from(cashShifts)
    .where(eq(cashShifts.id, movement[0].shiftId));

  if (shift[0]?.status === "closed") {
    return res.status(400).json({
      error: "No se puede anular un movimiento de un turno cerrado. Contacte a ad
    });
  }
}
// ... resto del endpoint sin cambios
```

Fix en el frontend: el botón de anular ya existe. Solo agregar una verificación visual:

```
// En la lista de movimientos, deshabilitar el botón si el turno está cerrado
// El turno actual siempre está abierto (es el currentShift), así que
// esto solo aplica en la vista de historial donde se ven turnos pasados
```

NOTAS TÉCNICAS

- `BarChart3`, `CreditCard` de `lucide-react` — verificar que estén importados o agregar
- `PAYMENT_METHOD_MAP` ya existe en el componente — reutilizar
- `formatCurrency` ya existe en el componente — reutilizar
- El endpoint `/api/reports/caja-unificada` ya existe — solo ampliar la respuesta con

porModulo

- El tab “Resumen del Día” reemplaza la necesidad de ir área por área para ver el estado general
- Sin cambios de DB, sin nuevas tablas